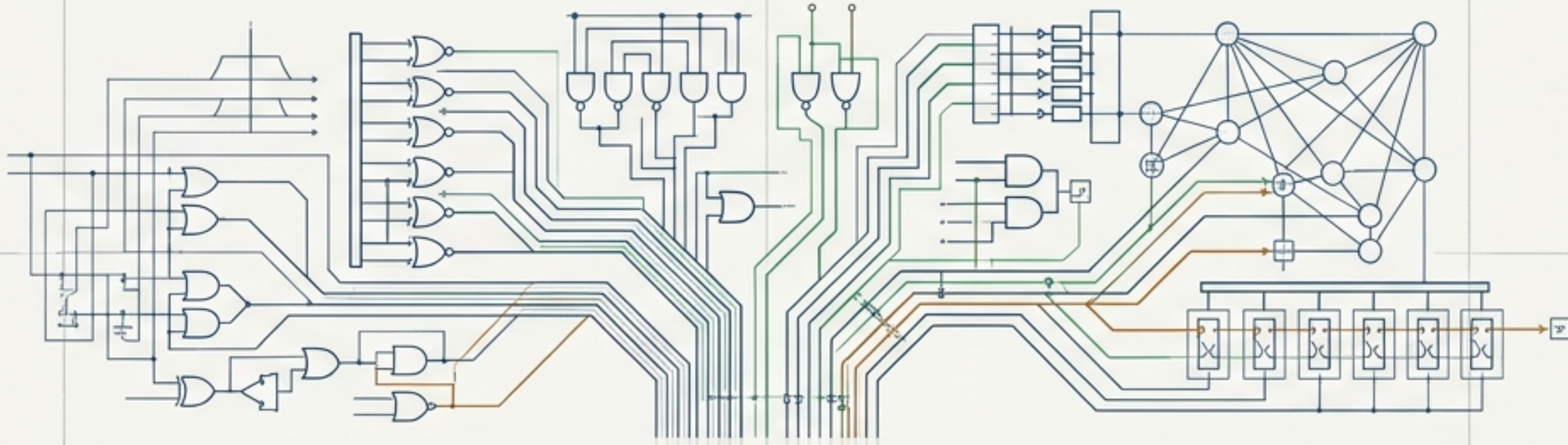




Verinin Görünmez Motoru: B ve B+ Ağaçları

Modern Sistemlerin Bilgiyi Düzenleme ve Bulma Mimarisi

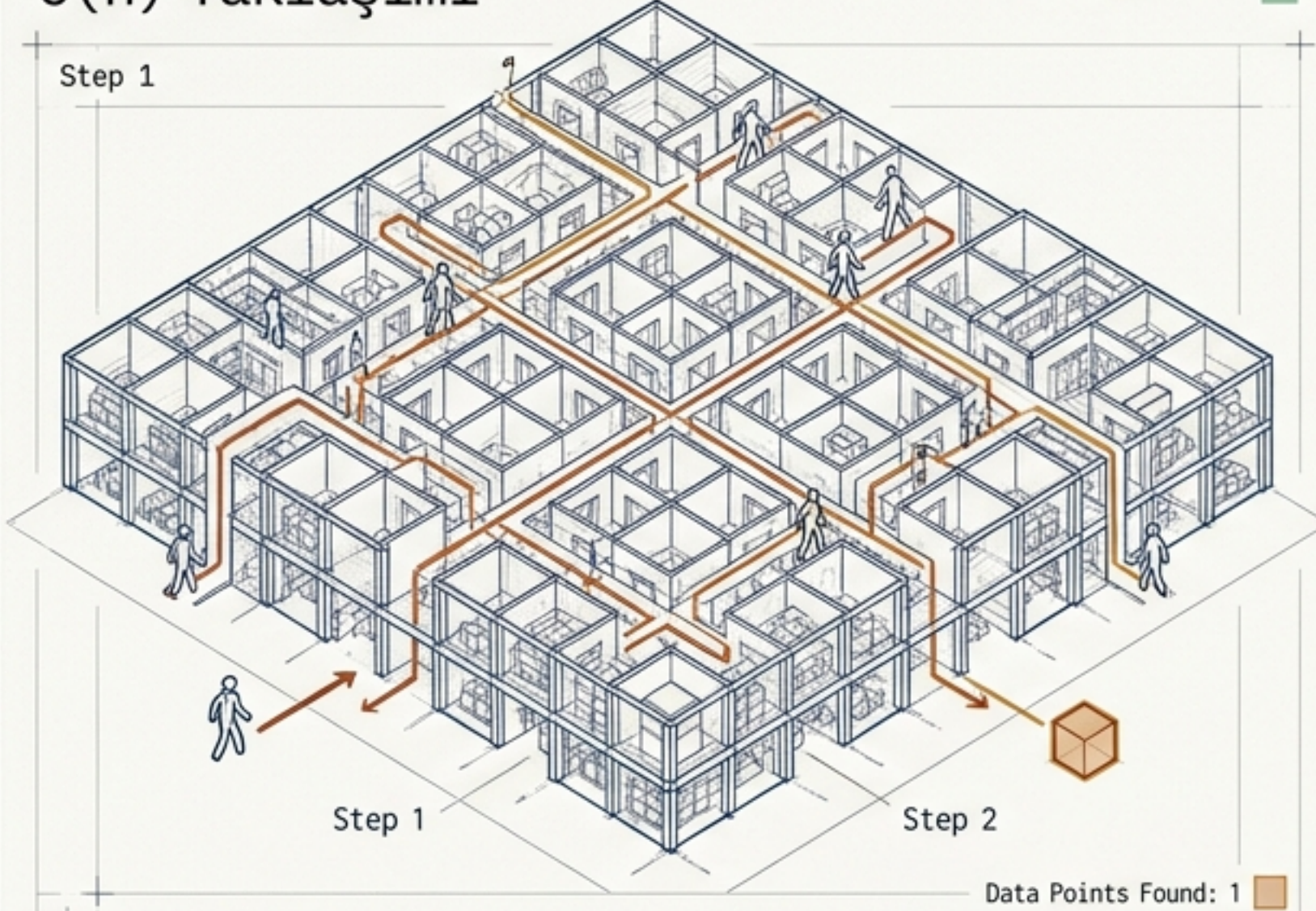


Hazırlayan
Ogün Yıldırım
Yazılım Mühendisi

Büyük Veri Problemi: Samanlıkta İğne Aramak

$O(n)$ Yaklaşımı

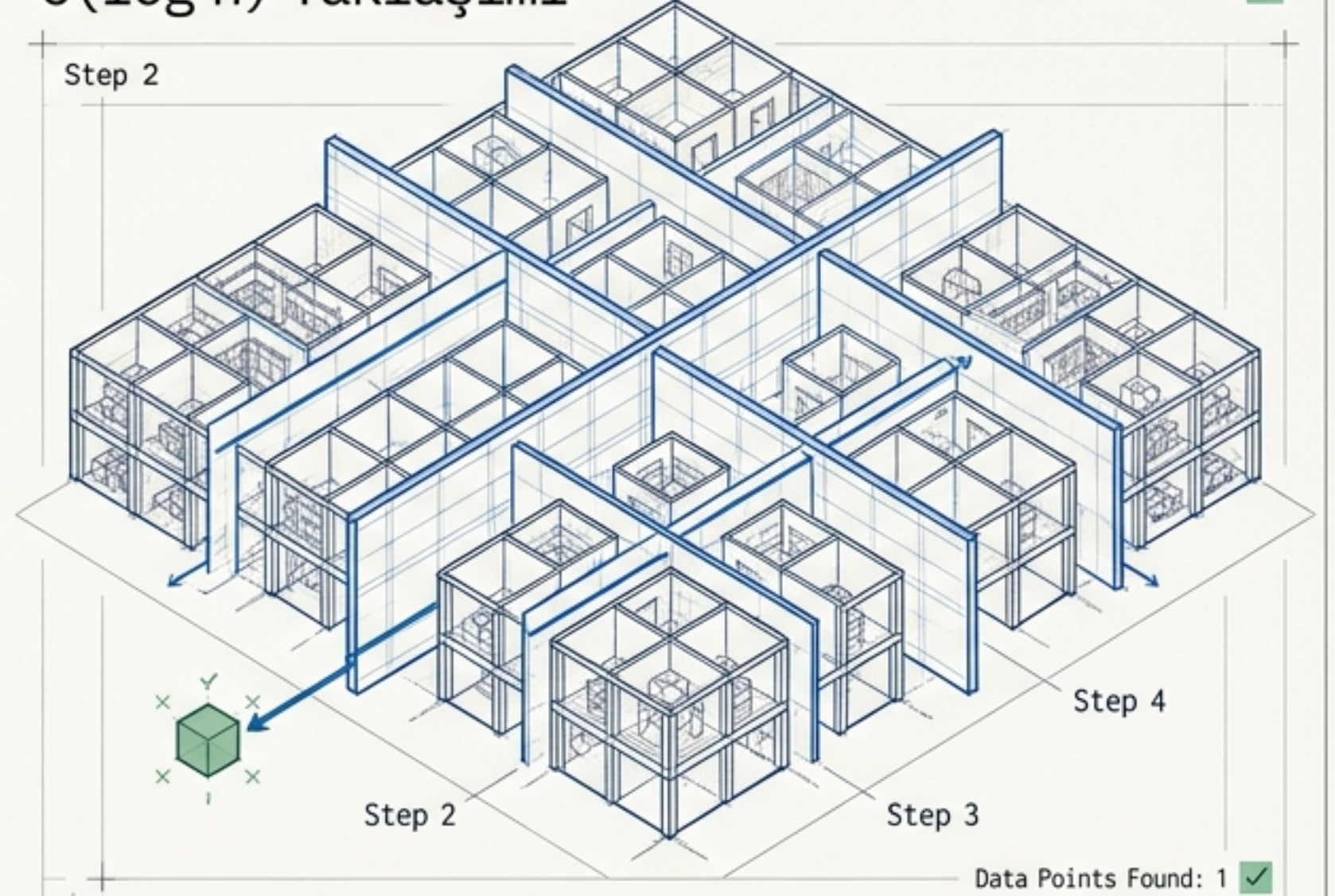
Data Points Found: 



Sıralı Tarama (Linear Scan). Veri tabanları veriyi düz bir dosyada tutsaydı, aradığınız tek bir satır için milyarlarca kaydın tek tek okunması gerekirdi. Veri büyüdükçe süre de aynı oranda uzar.

$O(\log n)$ Yaklaşımı

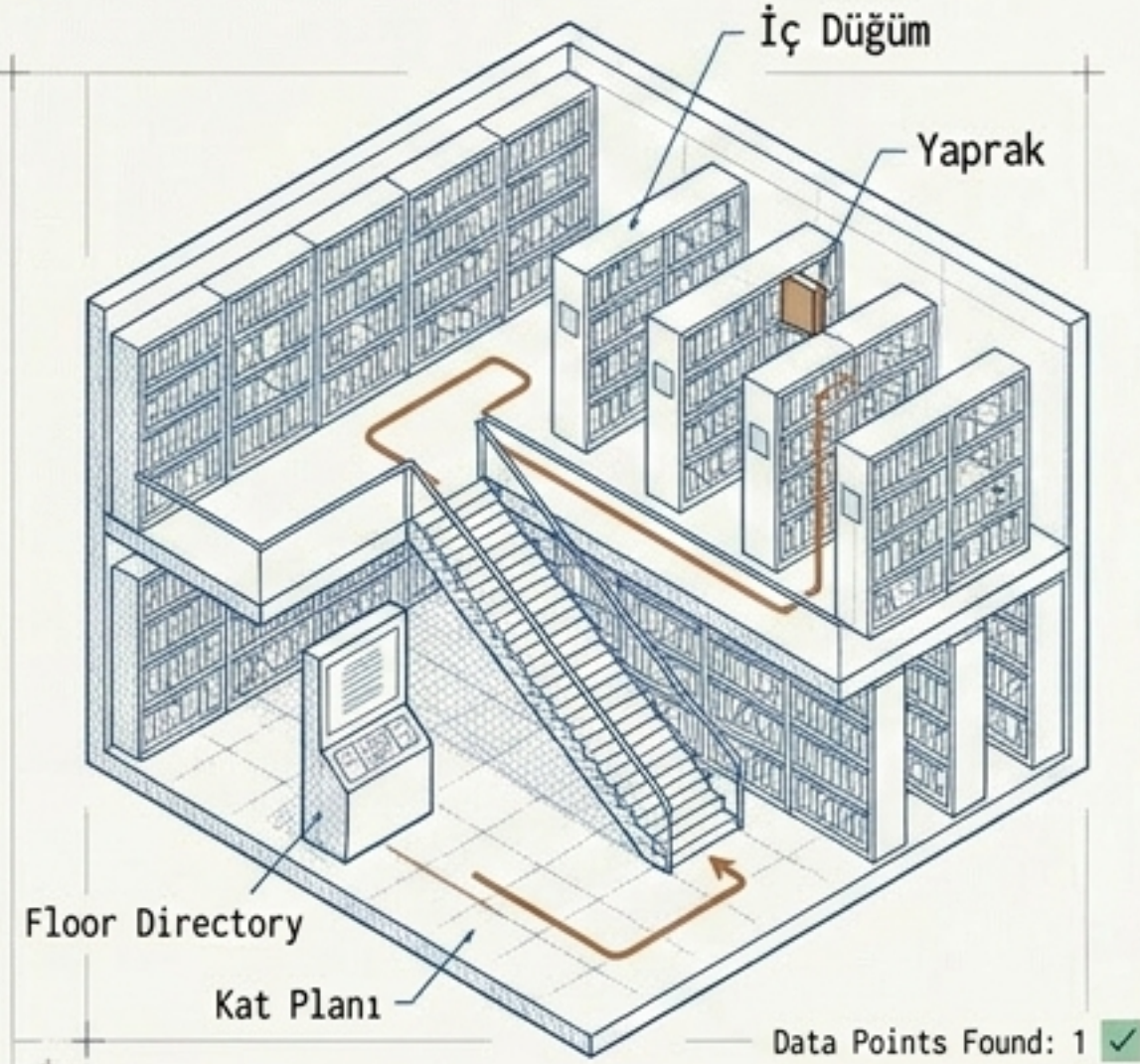
Data Points Found: 



Logaritmik Arama (Logarithmic Search). Veri havuzunu her adımda daraltarak, milyarlarca kayıt içinden aradığınızı sadece 3-4 adımda bulma yeteneği. B ve B+ ağaçlarının varoluş amacı.

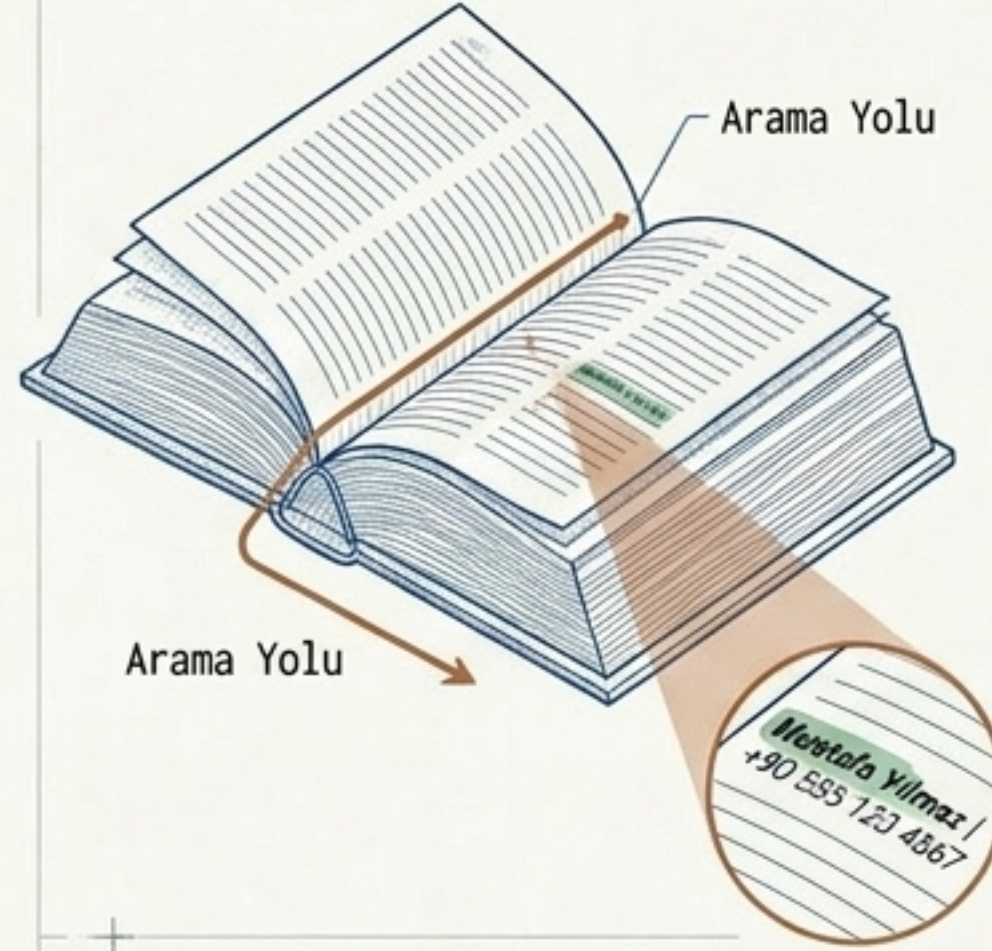
Fiziksel Dünyanın Arama Algoritmaları

Karmaşık veri yapıları, aslında günlük hayatta bilgiye ulaşmak için kullandığımız somut sistemlerin dijital yansımalarıdır.

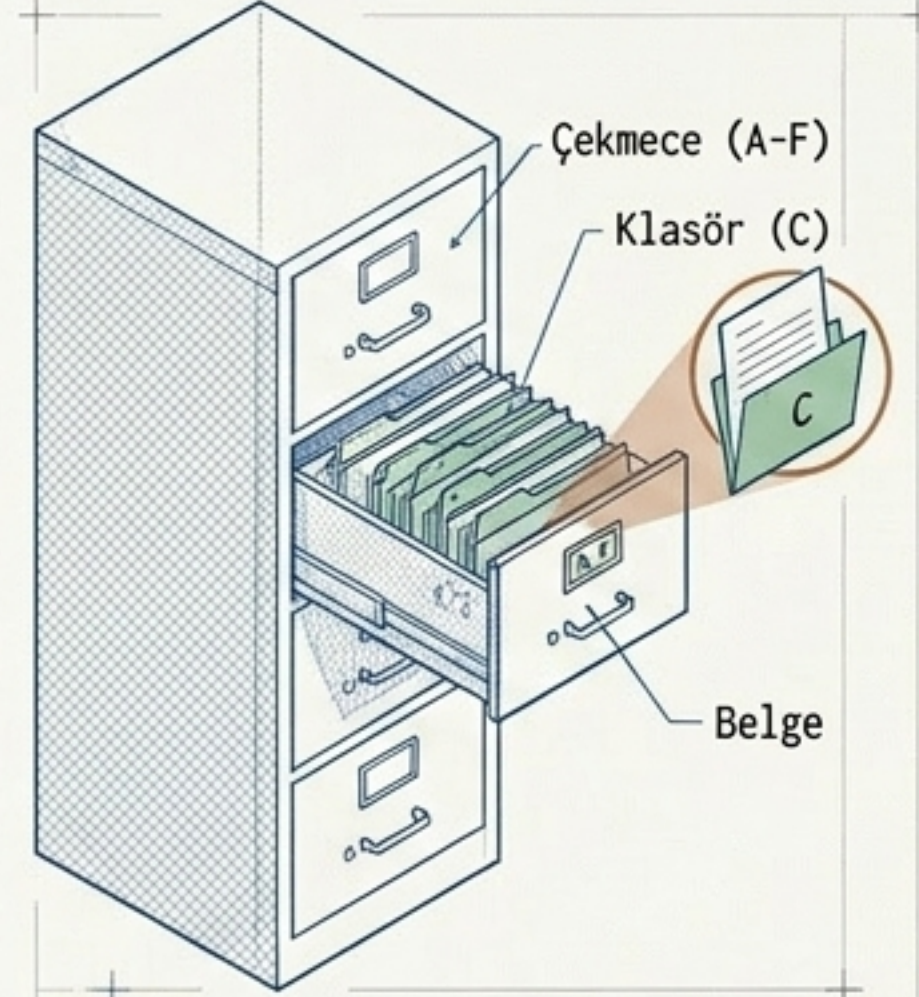


Kütüphane Kat Planı:

Aradığınız kitabı bulmak için her rafı gezmezsiniz. Ana kat planı (Kök) sizi doğru koridora (İç Düğüm), koridor ise doğru rafa ve kitaba (Yaprak) yönlendirir.

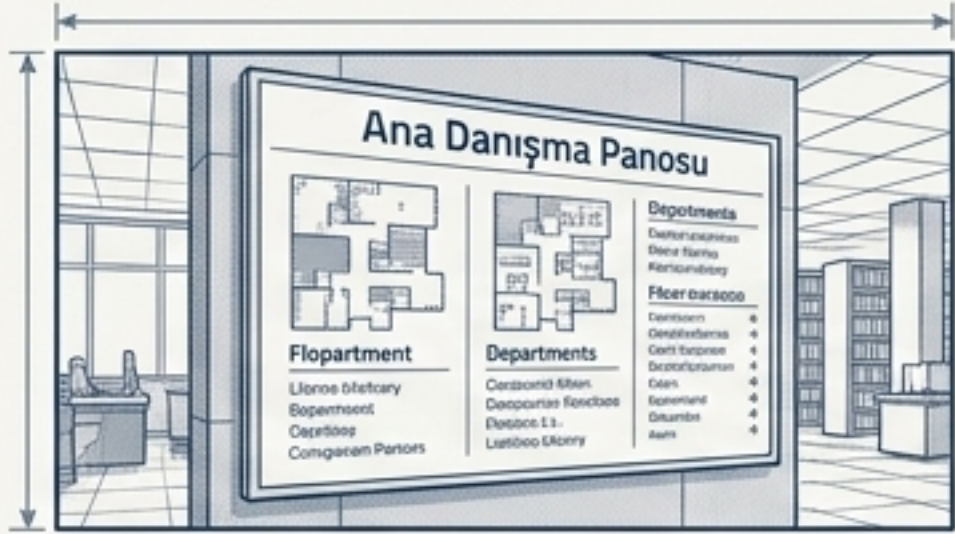


Telefon Rehberi: "Mustafa" ismini ararken baştan başlamazsınız. Önce 'M' harfine, sonra 'Mu' hecesine atlar, arama alanını sürekli daraltırsınız.



Dosya Dolabı: Doğru çekmeceyi (A-F arası) bulmak, ardından doğru klasörü (C harfi) seçmek, hedefe ulaşmak için gereken adım sayısını minimize eder.

Fiziksel Mimariyi Dijitale Çevirmek



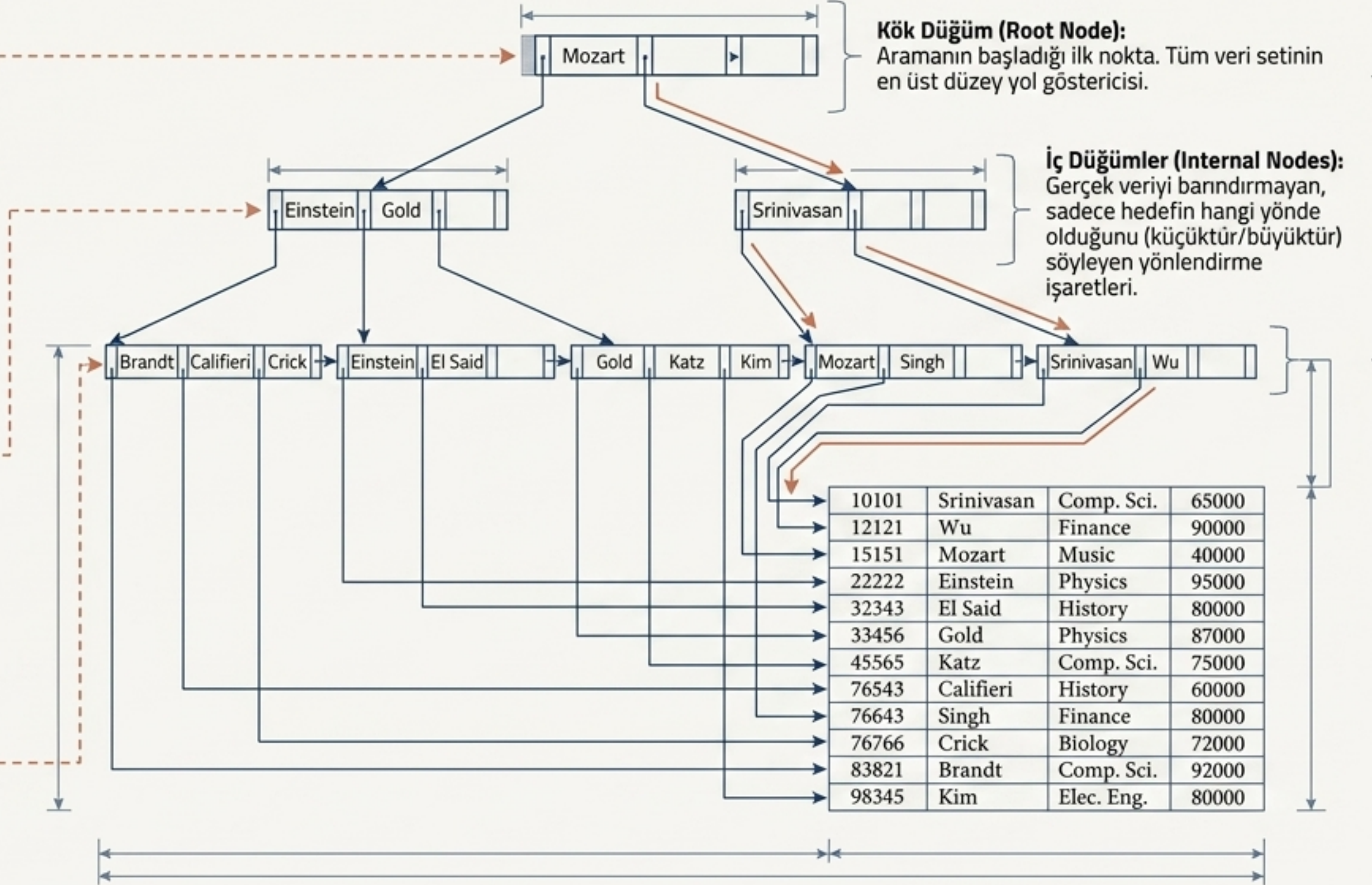
Ana Danışma Panosu



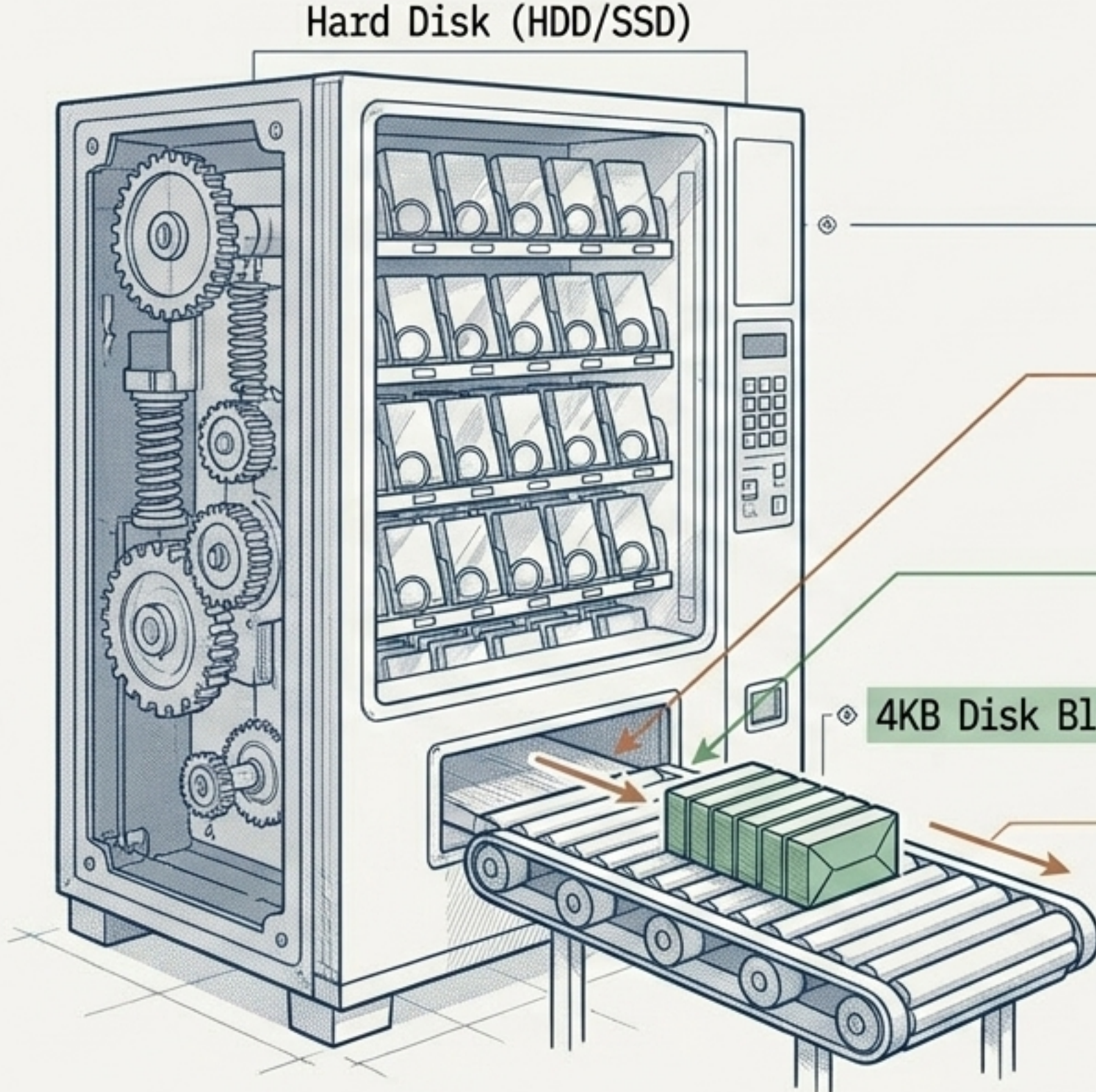
Koridor ve Raf Tabelaları



Raflardaki Kitaplar



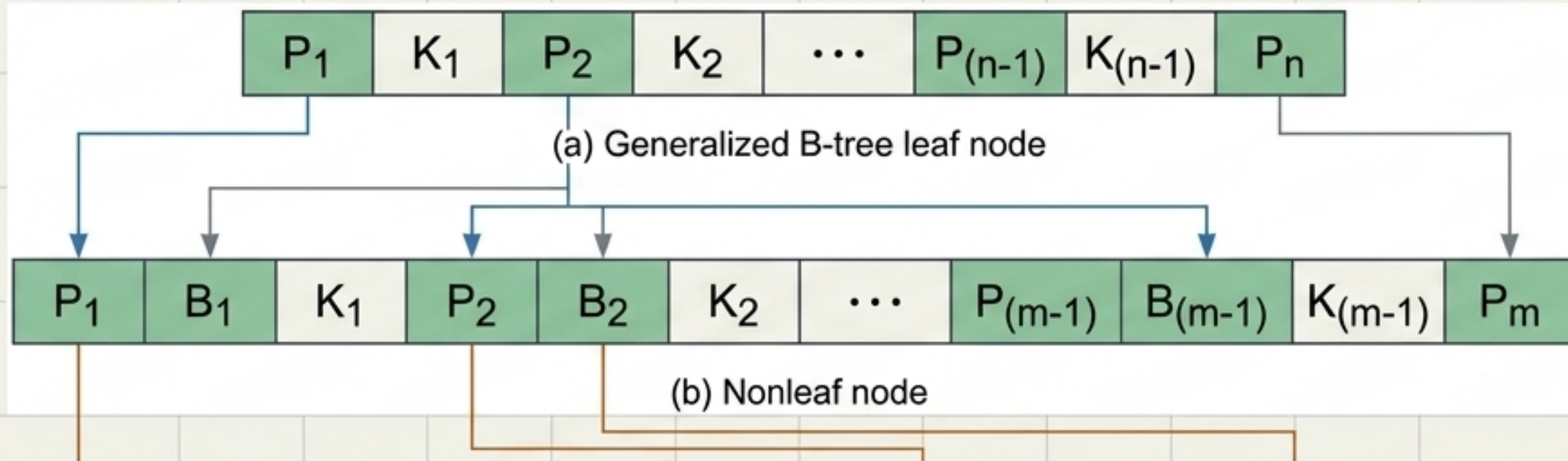
Disk G/Ç (I/O) ve Otomat Makinesi Analojisi



Neden klasik ağaçlar (Binary Trees) yerine B-Ağaçlarını kullanıyoruz?

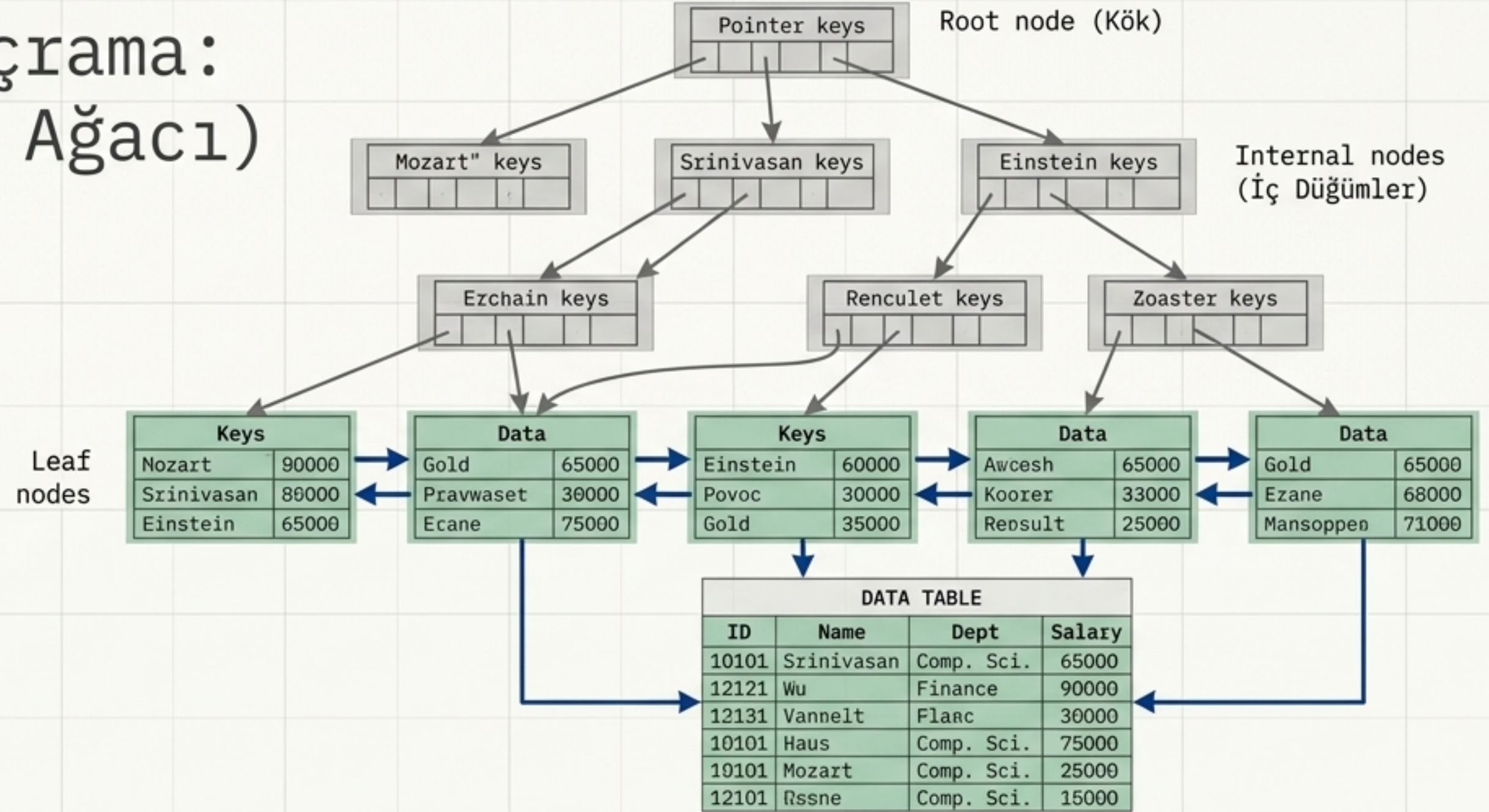
- **Darboğaz (Bottleneck):** Bir bilgisayarın en yavaş işlemi diskten veri okumaktır (Disk I/O).
- **Paket Sistemi:** Sabit diskler veriyi tek tek byte olarak değil, her zaman sabit bloklar (genellikle 4KB) halinde okur.
- **Otomat Makinesi:** Bir otomatın sadece 10'lu paketler halinde ürün verdiğini düşünün. Sadece 1 şekere ihtiyacınız olsa bile makine size 10'lu paketi verir.
- **Geniş ve Sığ Ağaçlar:** B/B+ ağaçlarının her bir düğümü tam olarak bir disk bloğuna sığacak şekilde tasarlanmıştır. Amaç, aşağı doğru (derin) değil, yanlara doğru (geniş) büyüyerek disk okuma (I/O) gezilerini en aza indirmektir.

Temel Yapı: B-Tree (B-Ağacı)



- Veri Her Yerde: B-Ağacında arama anahtarları (search keys) ve bu anahtarlara ait gerçek veriler, ağacın kök ve iç düğümlerinde de bulunabilir.
- Erken Bulma İhtimali: Aranan veri bir iç düğümden ise, en alta inmeden veri bulunabilir.
- Dezavantaj: İç düğümler verinin kendisini de tuttuğu için, bir 4KB bloğuna daha az sayıda "yönlendirme anahtarı" sığar. Bu da ağacın daha çabuk derinleşmesine ve yavaşlamasına neden olur.

Evrimsel Sıçrama: B+ Tree (B+ Ağacı)



- **Veri Sadece Yapraklarda:** Gerçek veriler sadece en alt kattaki yaprak düğümlerde saklanır.

- **Daha Sığ ve Hızlı:** İç düğümler veri tutmadığı için, tek bir disk bloğuna yüzlerce yönlendirme anahtarı sığdırılabilir. Ağaç sığlaşır, arama hızı artar.

- **Yatay Bağlantılar (Linked List):** Yaprak düğümler birbirine çift yönlü zincir gibi bağlıdır. Bu özellik, modern veri tabanlarının en büyük gücüdür.

Karşılaştırma Matrisi:

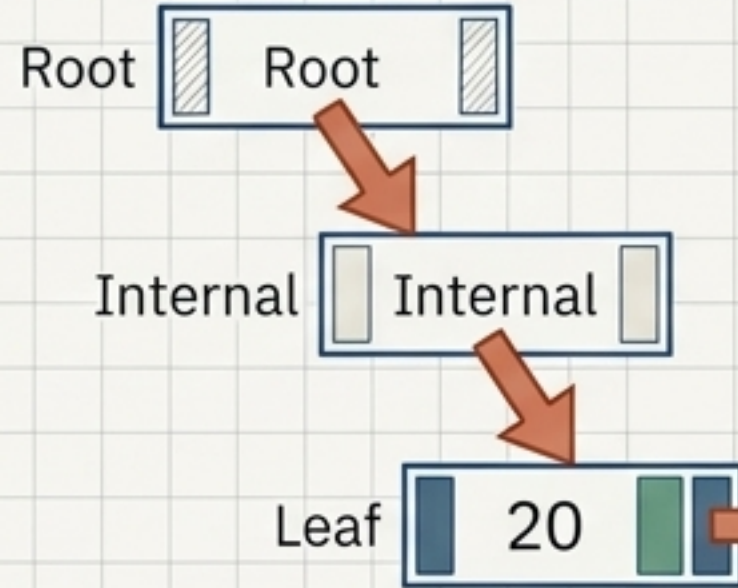
B-Tree vs. B+ Tree

Özellik	B-Tree	B+ Tree (Modern Standart)
Veri Depolama Yeri	Tüm düğümlerde (Kök, İç, Yaprak)	Sadece Yaprak düğümlerde
Ağaç Derinliği	Daha derin (İç düğümler veri tuttuğu için)	Daha sığ (İç düğümler daha fazla anahtar alır)
Aralık Sorguları (Range)	Çok Yavaş (Ağaç içinde sürekli in-çık gerektirir)	Çok Hızlı (Bağlı liste üzerinden yatay ilerler)
Hafıza (RAM) Verimliliği	Düşük	Yüksek (Sadece indeksler RAM'e yüklenebilir)
Kullanım Alanı	Nadir (Spesifik dosya sistemleri)	Dünyadaki neredeyse tüm popüler veri tabanları

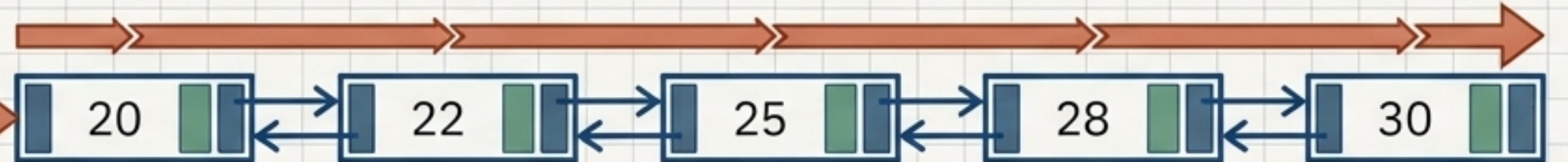
B+ Ağaçlarının Süper Gücü: Aralık Sorguları (Range Queries)

```
SELECT * FROM Kullanicilar WHERE Yas BETWEEN 20 AND 30
```

1. Dikey Düşüş ($O(\log n)$)



2. Yatay Tarama



B+ Ağaçlarının birbirine bağlı yaprakları sayesinde, bir veri aralığını bulmak (örn: belirli tarihler arası siparişler) için ağaçta tekrar yukarı çıkmanıza gerek kalmaz. Sadece başlangıç noktasını bulur ve zinciri takip edersiniz.

Günümüzde Ekosistemdeki Kullanım Alanları



Veri Tabanları (Databases):

İlişkisel (RDBMS) ve NoSQL veri tabanlarının varsayılan indeksleme altyapısı. Milyarlarca satırlık tablolarda anında arama sağlar.



Arama Motorları (Search Engines):

Web sayfalarından çıkarılan ters indekslerin (inverted index) disk üzerinde verimli bir şekilde saklanması ve hızlı erişimi.



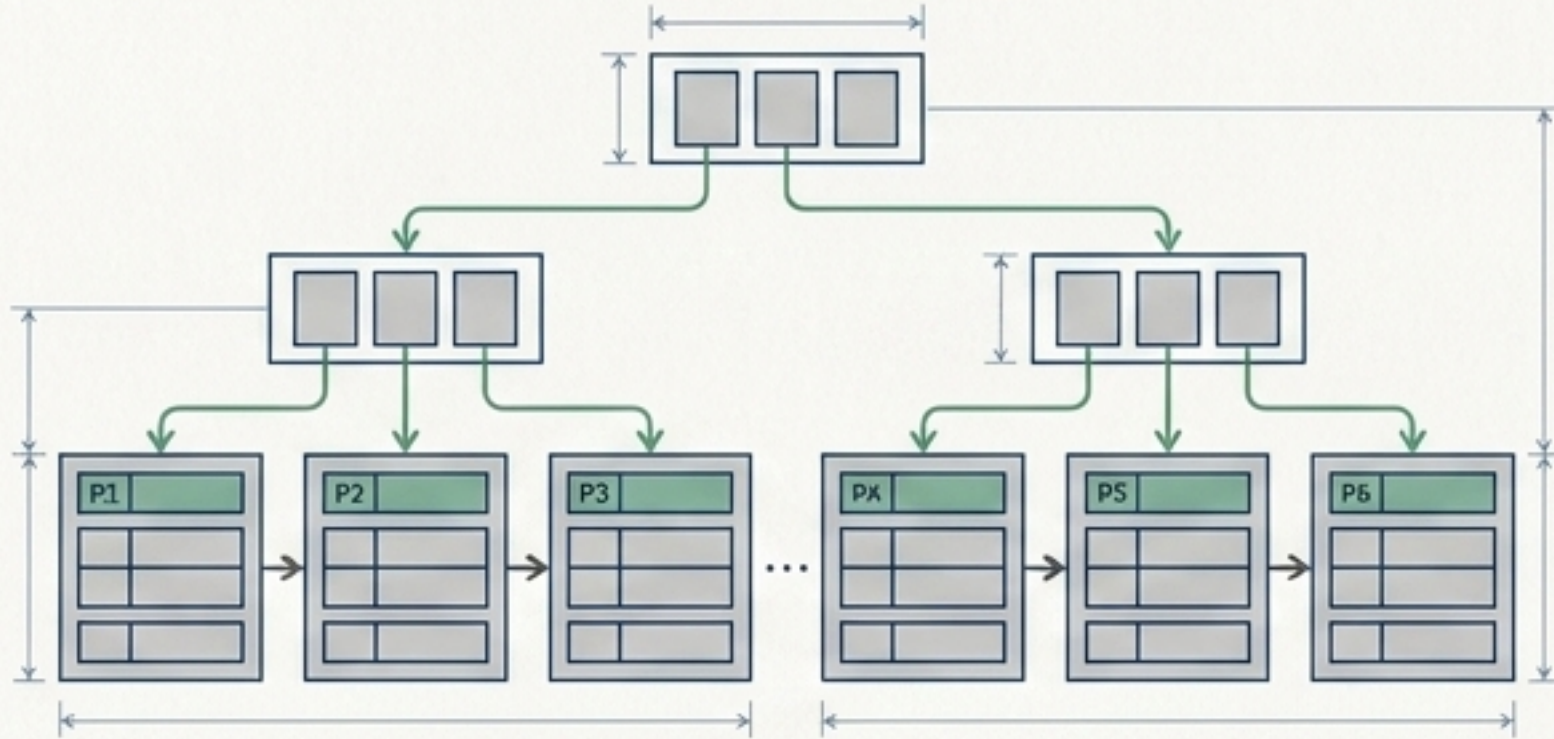
Dosya Sistemleri (File Systems):

NTFS, APFS ve Ext4 gibi sistemler, sabit diskinizdeki milyonlarca klasör ve dosyayı bulmak için B/B+ ağaçlarını kullanır.

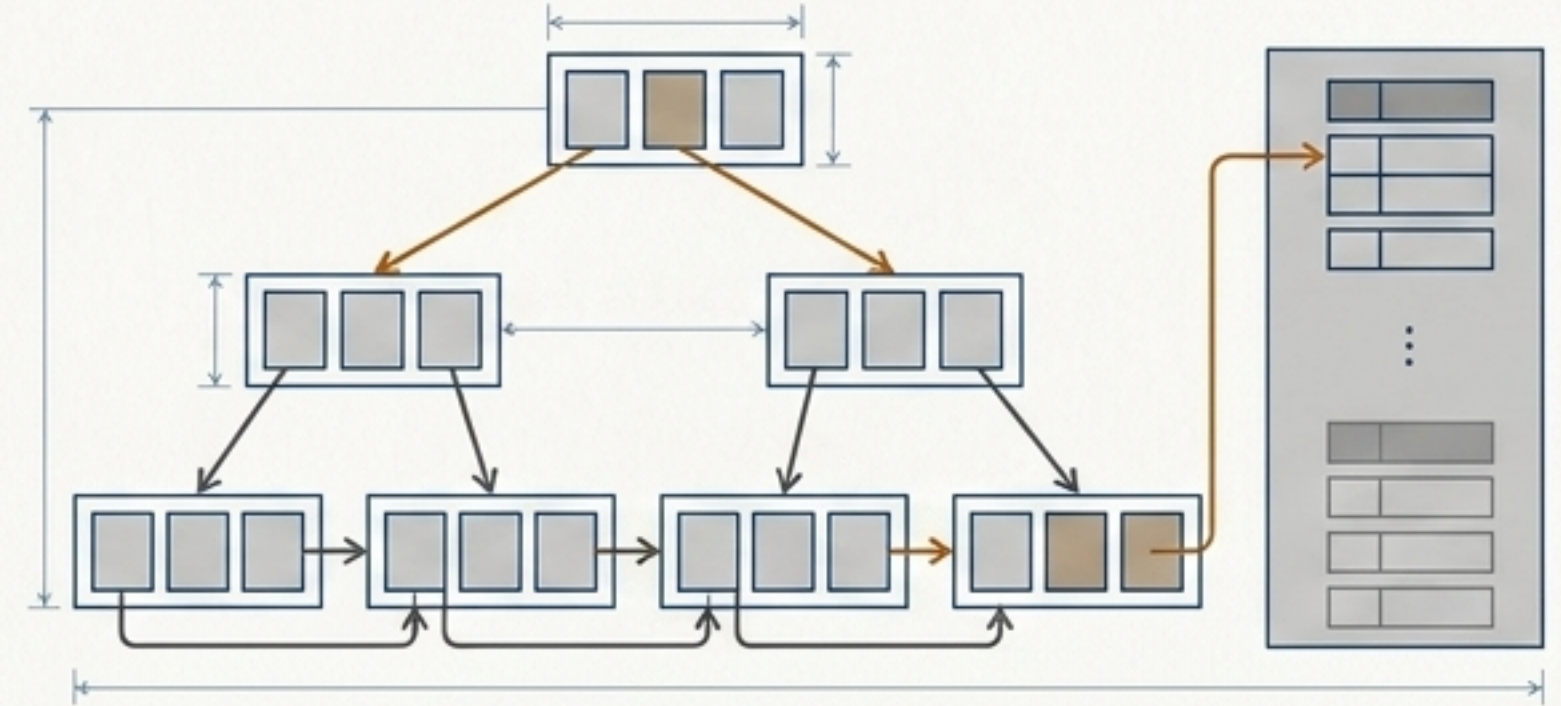
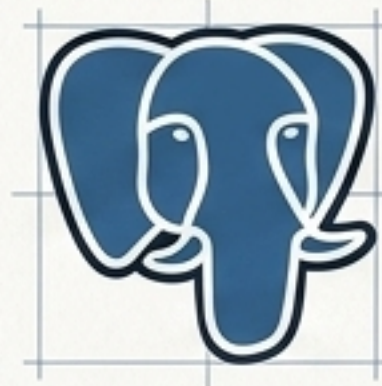


Büyük Veri (Big Data): Hadoop, Cassandra gibi dağıtık veri sistemlerinde, yığınla veri içindeki spesifik kayıt parçalarını hızlıca adreslemek.

Modern Titanlar: MySQL ve PostgreSQL



MySQL (InnoDB): InnoDB depolama motoru B+ Ağaçlarına o kadar bağımlıdır ki, tablonun tüm verisini doğrudan Primary Key (Birincil Anahtar) üzerine inşa edilmiş bir B+ Ağacının yapraklarında saklar (Clustered Index). Her tablo fiziksel olarak bir B+ ağacıdır.



PostgreSQL: Varsayılan indeksleme mimarisi olarak B-Tree varyantlarını kullanır. Mükemmel nokta atışı aramalar (point lookups) ve ORDER BY işlemleri için veriyi diskte sıralı tutma konusunda eşsiz bir kararlılık sunar.

Modern Titanlar: MongoDB (NoSQL)

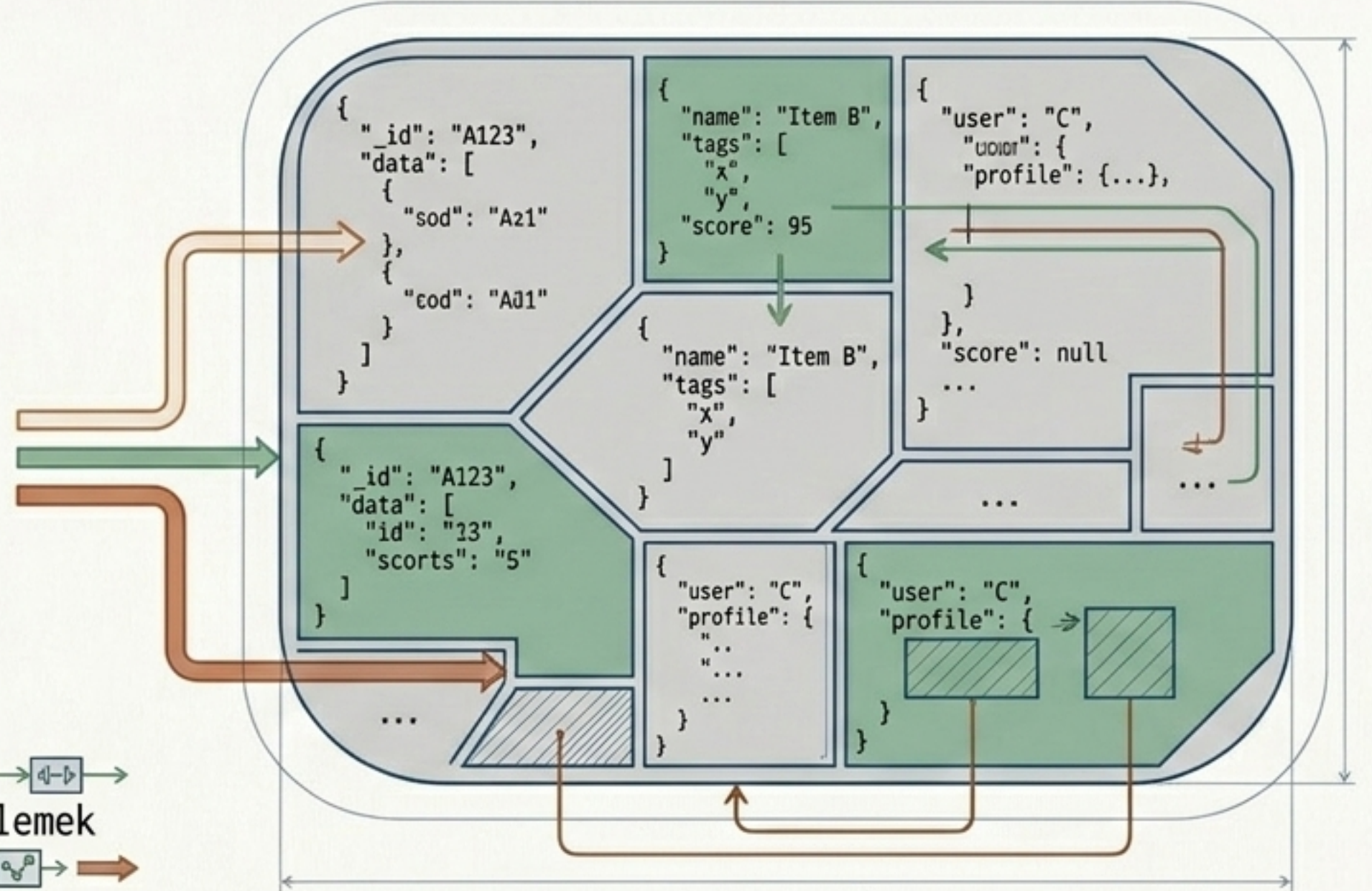
"B+ Ağaçları sadece sabit sütunlu ilişkisel tablolar için değildir."



- MongoDB'nin varsayılan depolama motoru olan **WiredTiger**, esnek NoSQL dokümanlarını indekslemek için B-Tree / B+ Tree yapılarını kullanır.



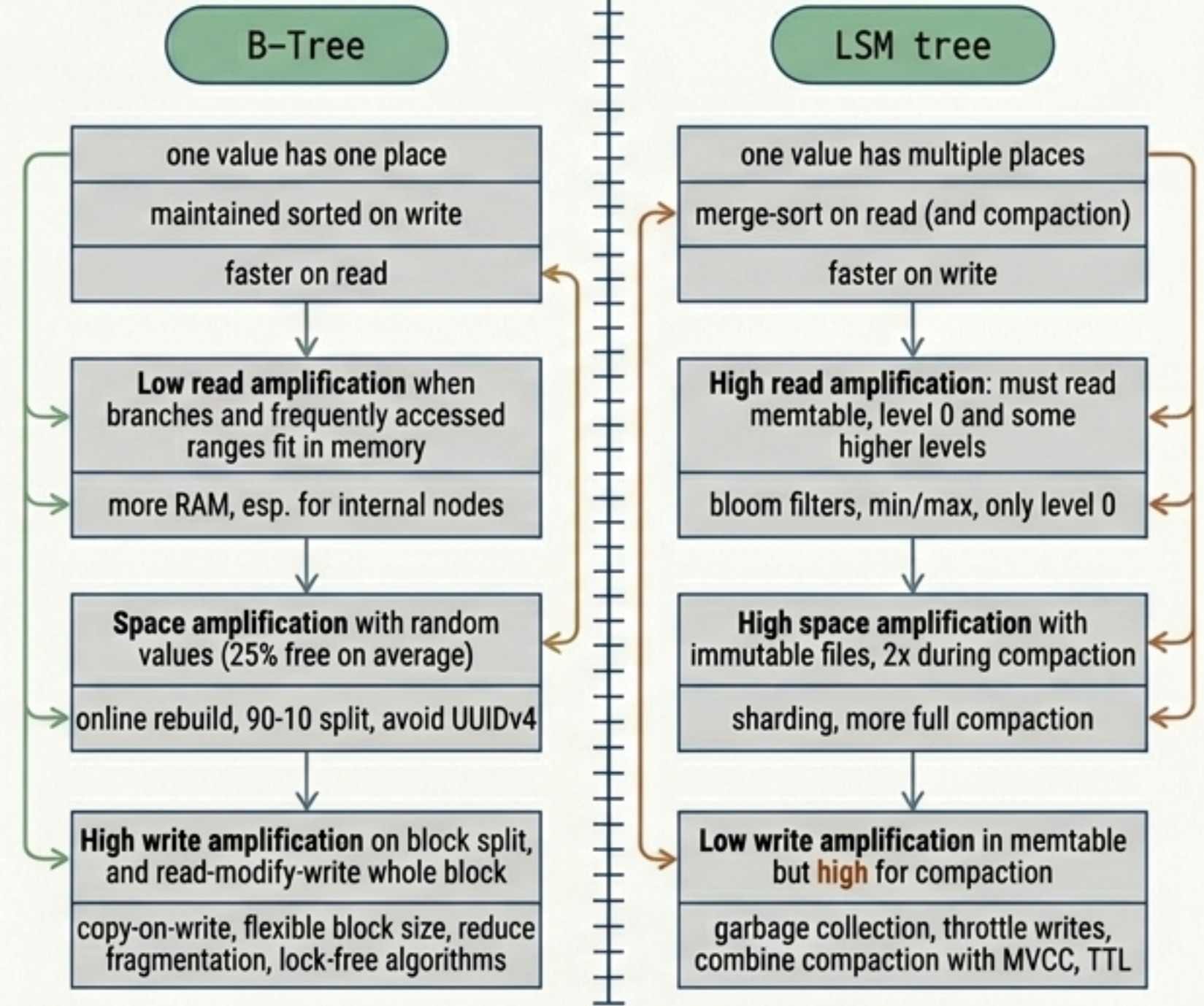
- **\$gte** (büyüktür), **\$lte** (küçüktür) gibi karmaşık aralık taramaları, arka planda tamamen B+ Ağacının mantığıyla çalışarak NoSQL'in esnekliğini disk verimliliğiyle birleştirir.



Büyük Veri Çağında Gelecek Ufku: Hayatta Kalacaklar Mı?

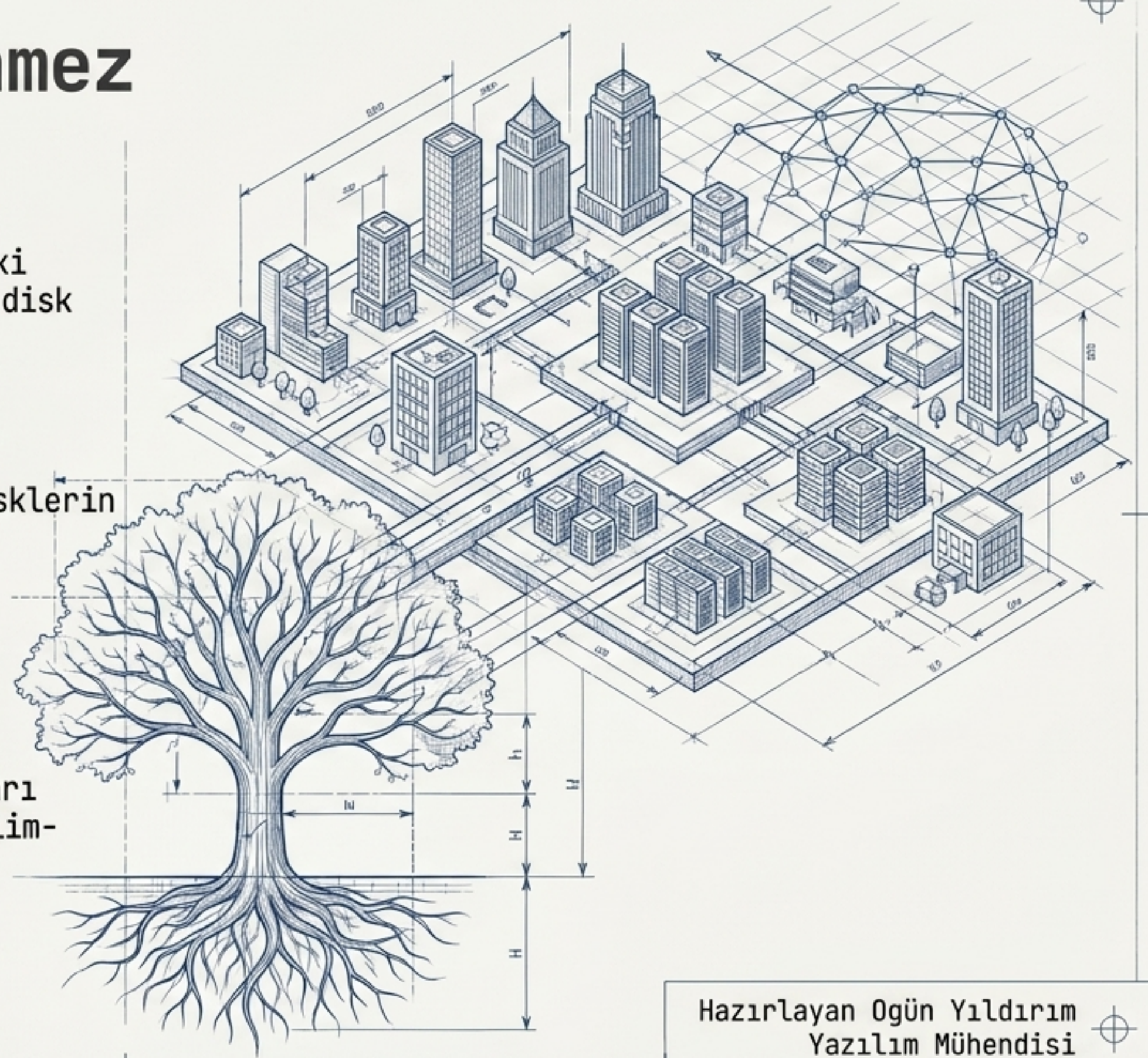
B ve B+ ağaçları onlarca yıllık geçmişine rağmen neden hala vazgeçilmez?

- **Fizik Kanunları Değişmedi:** Bulut sistemler ne kadar büyürse büyüsün, verinin fiziksel disk bloklarından (4KB) belleğe aktarılma maliyeti hala en büyük darboğazdır. Ağaç yapıları bu maliyeti optimize eden en mükemmel mimaridir.
- **Büyük Veride İhtiyaç:** Logaritmik ($O(\log n)$) ölçeklenme sayesinde veri 10 katına çıksa bile arama süresi sadece 1 birim artar.
- **Gelecek Vizyonu (LSM Tamamlayıcılığı):** Sürekli yüksek yazma (write-heavy) gerektiren bazı modern bulut sistemlerinde (RocksDB, Cassandra) LSM Ağaçları popülerleşse de, düşük okuma gecikmesi (low read amplification) ve çok yönlülük gerektiren genel sistemlerde B+ Ağaçları standart olmaya devam edecektir.



Özet: Verinin Görünmez Mimarları

1. **Düzensizliği Düzenler:** İnsan ölçeğindeki karmaşık sorguları, makine ölçeğindeki disk G/Ç verimliliğine çevirir.
2. **Analogdan Dijitale:** Kütüphane ve dosya dolaplarının binlerce yıllık fiziksel organizasyon mantığını, modern sabit disklerin mikro saniyelerine uyarlar.
3. **Modern Altyapının Kalbi:** MySQL'den MongoDB'ye, arama motorlarından dosya sistemlerine kadar dijital hayatımızın temel taşıdır.
4. **Zamana Meydan Okuyan Mimari:** B+ Ağaçları bir yazılım trendi değil, bilgisayar bilimlerinin sarsılmaz bir temel kanunudur. Geleceğin verisini de bu kökler taşıyacaktır.



Hazırlayan Ogün Yıldırım
Yazılım Mühendisi